

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment.

1. System Requirements

System Name: ResuFit: Precision CV Matching System

Architecture Style: Microservices

Functional Requirements

- submitting the CV
- viewing and editing the CV
- answering screening questions
- getting a rating for the submitted CV

Non-Functional Requirements

- Performance

Description: The system **MUST** respond within a reasonable time frame.

- Reliability

Description: The system **SHALL** be available at least 99.9% of the time.

- Security

Description: The system **MUST** protect user data and prevent unauthorized access.

- Privacy

Description: The system **SHALL** ensure that user data is handled in accordance with applicable laws and regulations.

- Scalability

Description: The system **SHOULD** be able to handle a large number of users and CVs without a significant performance degradation.

- Usability

Description: The system **MUST** be easy to use and navigate for all users.

- Maintainability

Description: The system **SHOULD** be easy to modify and update.

- Compatibility

Description: The system **MUST** be compatible with a variety of devices and browsers.

- Integration

Description: The system **SHOULD** be able to integrate with other HR systems and tools.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, con-

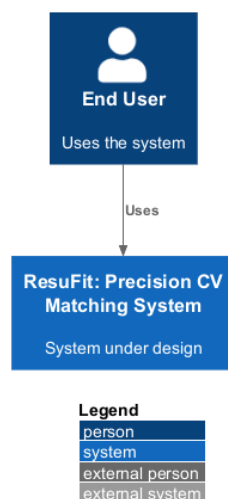
```
System ResuFit:PrecisionCVMatchingSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Microservices";
  Property source : String = "FALLBACK";
  Component APIGateway = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Route client requests";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
    Property type : String = "L7";
    Property healthChecks : Boolean = true;
  }
  Component UserService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Manage users";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component OrderService = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Manage orders";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component OrderDatabase = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Persist order data";
    Property state : String = "stateful";
    Property scaling : String = "vertical";
    Property consistency : String = "eventual";
    Property replication : String = "multi-node";
  }
  Attachment APIGateway.out to UserService.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
  }
  Attachment APIGateway.out to OrderService.in {
    Property delivery : String = "at-least-once";
```

```

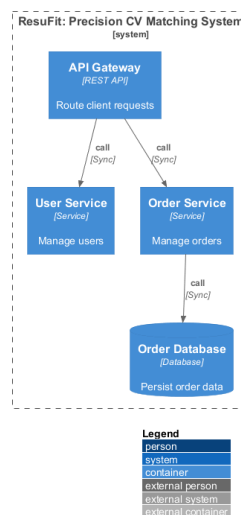
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment OrderService.out to OrderDatabase.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";
}

```

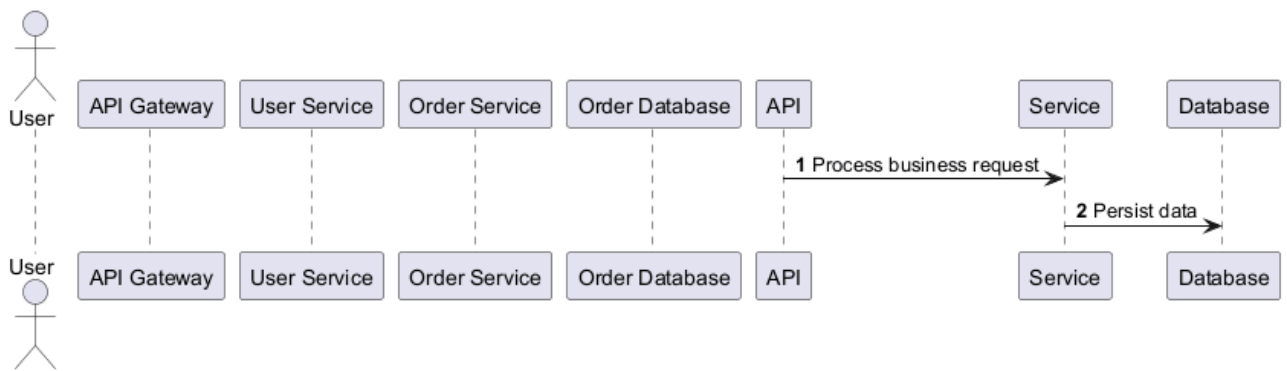
3. Context View



4. Logical View (C4 Container Diagram)



5. Process View (Runtime Interaction)



6. Physical View (Deployment Diagram)

